Rethinking Kolmaf-AI

From a desktop agent bridge to a seeded intelligent capability runtime

KoLmafia is the engine.

kolmaf-AI becomes the intelligence, evidence, instrumentation, and shared runtime layer.

A user, deterministic dispatcher, or agent can occupy the dealer seat.

Working architecture paper · 12 October 2026

Concepts synthesized from the current public Kolmaf-AI direction, the Combo + WaterSheep prototype, and proposed integrations with autoscend, garbo, and Oreo.

Kolmaf-AI is an independent experiment. The proposed integrations in this paper are architectural examples, not claims that upstream projects currently implement these runtime hooks.

Executive thesis

The earlier public Kolmaf-AI direction centered on a Linux-first desktop control desk connecting human intent, LLM reasoning, context tools, policy gates, KoLmafia, and post-action readback. Its safety premise was already strong: the model reasons and proposes; KoLmafia remains the game-aware local runtime and source of runtime truth. The new direction proposed here preserves that trust model while changing the unit of deployment.

> **The architectural rethink is not 'give AI more authority.' It is 'move the intelligence and instrumentation contract closer to each deterministic capability, then make the human or agent at the top replaceable.'**

Instead of treating cross-deployment as a family of large platform-specific bridges, Kolmaf-AI can become a common TypeScript/JavaScript runtime seed that scripts and plugins adopt around their own decision seams. Combo is the accidental reference implementation: a deterministic Beach Comb tool acquired pre-action model scoring, evidence ledgers, prediction/result binding, receipts, graph/matrix views, ML evaluation, and a user/agent-readable proof surface without surrendering execution authority.

This paper develops that idea through four increasingly rich examples: Combo as the prototype, autoscend as a deterministic ascension expert, garbo as a deterministic economic optimizer, and Oreo as a probabilistic optimizer whose own model can itself be measured and calibrated.

Contents

Executive thesis i

1 Context: what Kolmaf-AI was becoming 1
1.1 What remains unchanged . . . 1
1.2 What changes . . . 1

2 The architectural inversion 1
2.1 The dealer seat . . . 2
2.2 Why runtime seeding is different from bridge proliferation . . . 3
2.3 The Unreal / VCode analogy . . . 3

3 Combo: the accidental reference implementation 4
3.1 Runtime seeding from the Combo pattern . . . 4
3.2 OpenCode becomes a dealer adapter . . . 5
3.3 The operational conversation remains honest . . . 5

4 Case study: autoscend as an instrumented ascension engine 6
4.1 Wrap the expert, do not replace it . . . 6
4.2 Routing becomes a measurable decision . . . 7
4.3 Decision seams below whole quests . . . 7
4.4 Every ascension becomes an experiment stream . . . 8
4.5 Progressive trust, not a sudden autonomy jump . . . 9
4.6 Player variability becomes contextual evidence . . . 10
4.7 The deterministic system becomes teacher, baseline, and experiment generator 10
4.8 Graph / matrix / data for ascension debugging . . . 11
4.9 Failure receipts instead of giant-log archaeology . . . 11
4.10 autoscend summary . . . 12

5 Case study: garbo as an ML wind tunnel for KoL economics 12
5.1 Wrap the deterministic economist . . . 13
5.2 Natural decision seams: 'is this worth doing?' . . . 13
5.3 Prediction versus realized value . . . 14
5.4 valueOfAdventure as a measurement target . . . 14
5.5 Repeated economic decisions are excellent training/evaluation material . . . 15
5.6 Embezzler chains as a strategy experiment . . . 15
5.7 Familiar and equipment choices: evaluate the optimizer, not replace it . . . 16
5.8 Diet becomes a shadow experiment . . . 16
5.9 Decision receipts make complex failures legible . . . 17
5.10 Garbo's task engine already resembles a capability envelope . . . 17
5.11 Economic graph / matrix / data . . . 18
5.12 Detect valuation drift without letting ML override . . . 19
5.13 ML as a spading instrument . . . 19
5.14 The dealer can remain thin . . . 19
5.15 garbo summary . . . 20

6 Case study: Oreo as a calibration laboratory 20
6.1 Oreo already has purpose-built strategy models . . . 21
6.2 Instrument the optimizer itself . . . 21
6.3 A nearly perfect experiment boundary . . . 21

6.4 Shadow all strategies on one frozen state . . . 22
6.5 Lambda becomes a measurable policy parameter . . . 23
6.6 Measure model assumptions instead of replacing them . . . 23
6.7 WaterSheep as a model-of-the-model . . . 24
6.8 Visibility becomes an information-economics experiment . . . 24
6.9 Dynamite becomes a cost-substitution experiment . . . 25
6.10 Offline and live worlds can share the same policy contract . . . 25
6.11 The mine itself is a proof surface . . . 25
6.12 Calibration, not single-event correctness . . . 26
6.13 Model residuals can become deterministic knowledge . . . 26
6.14 Structured explainability becomes provenance . . . 27
6.15 The dealer remains a reader/caller, not a miner implementation . . . 28
6.16 Oreo summary . . . 28

7 The common runtime contract 29
7.1 The invariant loop . . . 29
7.2 A minimal capability envelope . . . 29
7.3 FACT, MODEL, POLICY, ACTION, OUTCOME . . . 30
7.4 WaterSheep should be replaceable . . . 30
7.5 Runtime seed versus domain adapter . . . 30

8 Migration path from the public desktop direction 30
8.1 1. Preserve the current trust model . . . 30
8.2 2. Treat Combo as the reference seed . . . 31
8.3 3. Add read-only/shadow adapters to mature tools . . . 31
8.4 4. Standardize dealer discovery . . . 31
8.5 5. Let OpenCode arrive late . . . 31
8.6 6. Promote learned authority only by evidence, if ever . . . 31

9 What this reframing buys 31

10 Closing thesis 32

11 References and source notes 32

12 Concept index 33

1 Context: what Kolmaf-AI was becoming

The current public Kolmaf-AI page describes an experimental desktop layer for connecting LLM reasoning to KoLmafia without pretending the model is the game client. It presents the system as a control desk between a person, an LLM, and KoLmafia, with the trust boundaries kept visibly separate. The public formulation is concise: 'LLM intelligence, KoLmafia runtime truth.'

That direction is Linux-first. The desktop is the workspace, context tools feed reasoning, policy and confirmation gates bind proposed actions to state, KoLmafia performs approved interactions, and a readback step verifies what actually changed. Around it sits a constellation of helper projects for sandboxing, typed game data, memory, skills, bounded documents/evidence, local navigation, retrieval, ASH analysis, and actor/state control.

The prior architecture therefore already solved an important conceptual problem: model confidence is not permission. The rethink in this document changes a different layer - how capabilities are packaged, instrumented, and exposed across deployment contexts.

1.1 What remains unchanged

- KoLmafia remains the authoritative live runtime for game state and mutations.
- LLM or learned-model output is proposal material, not permission.
- Evidence, retrieval, memory, and UI do not silently become execution authority.
- Mutation should remain bounded, reviewable, state-bound, and independently verifiable.
- Fallback paths must not become bypasses around the intended authority boundary.

1.2 What changes

Earlier deployment emphasis	Rethought deployment emphasis
Desktop/control desk as the main integration center	Seeded capability runtime as the common integration center
Cross-platform work expressed as multiple runtime/bridge lines	Cross-deployment expressed as a portable capability/evidence contract
Agent connected through a broad control plane	Agent or user occupies a thin, replaceable dealer seat
Helper projects connected around the desktop	Mature scripts/plugins gain the same instrumentation substrate in-place
Reasoning about a large live system	Reasoning over typed capability envelopes: state, candidates, decision, result, receipt

Source context

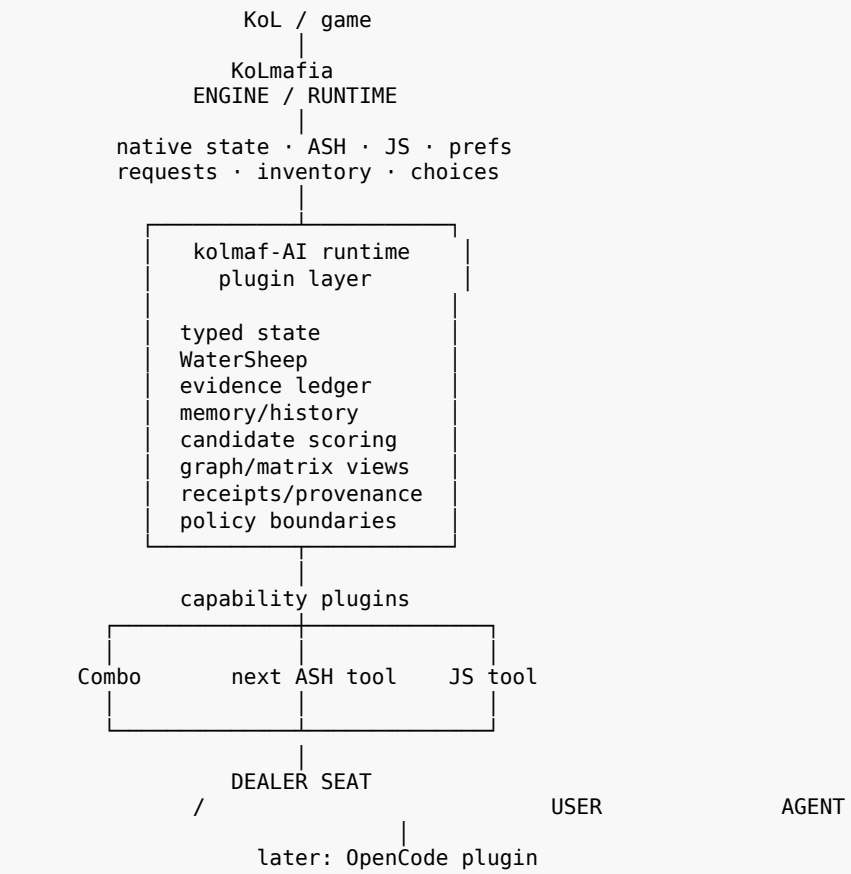
The prior-direction summary above is based on the public Kolmaf-AI project page as viewed on 12 October 2026. The page describes a Linux-first, human-gated desktop architecture, KoLmafia as runtime truth, proposal-oriented model use, post-mutation readback, and a multi-tool ecosystem. See the references section for the project URL.

2 The architectural inversion

The mistake is to think of Kolmaf-AI as the thing that plays KoL. A cleaner model is to treat KoLmafia as the engine, in the same sense that a game engine owns world state, native

execution, and core runtime behavior. Kolmaf-AI then becomes an intelligent plugin layer: typed state, learned inference, evidence, memory, instrumentation, policy boundaries, receipts, and visualization around capabilities that already know their domains.

Core engine / runtime / dealer stack



2.1 The dealer seat

The empty dealer seat is deliberate. The architecture does not assert that AI controls everything. It asserts that the system can expose a machine-readable, instrumented set of capabilities while leaving the identity of the caller as a deployment choice.

> **Who sits in the dealer seat is a deployment choice.**

The dealer can be a player clicking manually, a deterministic dispatcher, an OpenCode plugin, a future planner, or a hybrid in which an agent proposes and a user approves. The dealer chooses which supported capability to invoke; it does not need to reproduce the capability's domain mechanics.

Role separation

```
KoLmafia       = world/runtime authority
kolmaf-AI      = intelligence + instrumentation substrate
Combo/etc.     = domain capabilities
WaterSheep     = learned inference primitive
Dealer         = chooses which capability to call
```

```
Agent/User     = occupies or supervises dealer seat
```

2.2 Why runtime seeding is different from bridge proliferation

Cross-deployment no longer needs to mean reproducing the entire integration as a Linux bridge, a Windows bridge, a browser bridge, an MCP bridge, and a Python bridge. The host/runtime adapters can remain thin while the same capability contract is seeded into domain tools.

Cross-deployment as a contract, not a duplicated control plane

```
OLD EMPHASIS

kolmaf-AI
   │
   ├── bridge A → tool
   ├── bridge B → tool
   └── bridge C → tool


RUNTIME-SEED EMPHASIS

                kolmaf-AI runtime seed
              /          │                    plugin A    plugin B     plugin C
            │            │            │
       native host   native host  native host

          orchestration sits ABOVE them
```

2.3 The Unreal / VCode analogy

```
Unreal Engine
├─ renderer
├─ physics
├─ world state
├─ actors
└─ native execution
       │
       ▼
VCode / intelligent plugin layer
├─ learned analysis
├─ semantic state
├─ suggestions
├─ visualization
├─ tooling
└─ instrumentation
       │
       ▼
developer / agent

versus

KoLmafia
├─ KoL session
├─ inventory/state
├─ ASH/JS
├─ requests
├─ choices
└─ native execution
       │
       ▼
kolmaf-AI runtime
```

```
├─ WaterSheep
├─ semantic/evidence state
├─ scoring
├─ memory
├─ visualization
├─ receipts
└─ instrumented capabilities
       │
       ▼
player / agent
```

This analogy shifts the center of gravity. KoLmafia is not something the AI replaces. It is the engine the intelligent layer plugs into. The top-level user or agent becomes analogous to a developer using the engine through a rich plugin/tooling layer.

3 Combo: the accidental reference implementation

Combo began as a deterministic beach-combing script. Adding a tiny local WaterSheep loop did not merely produce an AI suggestion feature. Following the evidence requirements forced the tool to grow the surrounding machinery needed to make learned suggestions scientifically and operationally useful.

What Combo became

```
native KoLmafia state
      ↓
domain-specific deterministic tool
      ↓
shared TS/JS intelligence harness
      ↓
WaterSheep shadow inference
      ↓
FACT / MODEL / POLICY separation
      ↓
prediction + actual-result binding
      ↓
receipts / state / ML history
      ↓
interactive proof UI
      ↓
user/agent-readable capability
```

At that point Combo was no longer just a script with ML bolted on. It had become an instrumented capability. The runtime could preserve what was known before an action, what the model predicted, what deterministic policy did, what KoLmafia actually executed, and what happened afterward.

3.1 Runtime seeding from the Combo pattern

```
Combo
   + runtime seed
   = intelligent/instrumented beach capability

Loopstar-ish route tool
   + runtime seed
   = intelligent/instrumented routing capability

Puzzle solver
   + runtime seed
   = intelligent/instrumented puzzle capability
```

```
Combat tool
   + runtime seed
   = intelligent/instrumented combat capability
```

Each script retains ownership of the mechanics in its domain. The common runtime supplies a language for state, evidence, model scoring, deterministic policy, receipts, history, run control, and visualization.

3.2 OpenCode becomes a dealer adapter

```
OpenCode
   │
   │ discover capabilities
   │ inspect state/evidence
   │ request proposal
   │ call authorized capability
   ▼
kolmaf-AI dealer interface
   │
   ├─ Combo
   ├─ combat
   ├─ quests
   ├─ puzzles
   └─ whatever else is installed
```

An eventual OpenCode integration no longer needs to be the giant KoL bridge. It can be thin because the hard domain work stays inside the capability that already understands it.

A thin dealer API

```
list_capabilities()
inspect("combo")
propose("combo", state)
execute("combo", approved_request)
read_receipt(...)
```

3.3 The operational conversation remains honest

```
Dealer:
"Combo, assess/run this."

Combo:
"I know how Beach Comb works."

Runtime:
"Here is the evidence/model/policy state."

KoLmafia:
"I am the authority that actually knows
and changes game state."
```

Kolmaf-AI becomes an intelligent plugin runtime for KoLmafia, with composable instrumented capabilities and a replaceable user/agent dealer layer.

3.4 Combo field proof: live relay + frozen artifact

The Combo prototype now has two complementary proof surfaces: an in-situ KoLmafia relay view for operators and a frozen HTML artifact for post-run inspection. The relay view demonstrates deployment context; the frozen artifact carries the reproducible run record.

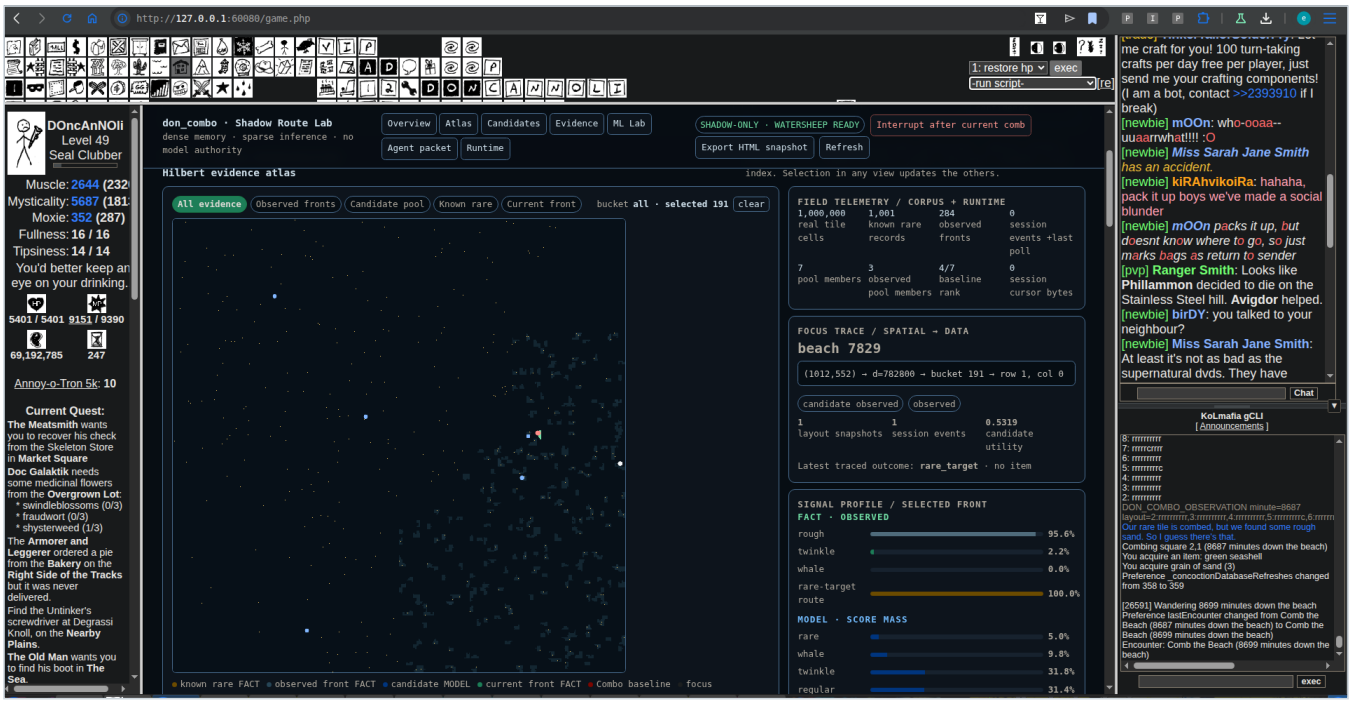


Figure 3-A. Live KoLmafia relay view - Shadow Route Lab / Hilbert evidence atlas.
The screenshot is interface evidence, not the event record for the buckle hit. It shows the runtime embedded in the KoLmafia relay alongside the game UI while retaining the visible “SHADOW-ONLY” posture.

PROVENANCE / WHAT EACH SURFACE PROVES

Live screenshot	KoLmafia relay integration and operator-facing proof UI; no claim that the selected beach shown is event #244.
Frozen HTML	don-combo-html-snapshot/v0.6; exported 2026-10-12T09:34:25.448Z; runtime generated 09:34:19.114Z.
Run envelope	200 requested / 200 completed; ML events 157 -> 357; delta 200; trace span complete = true.
Authority	Snapshot declares authority = none / static evidence only. WaterSheep remains shadow inference, not execution authority.

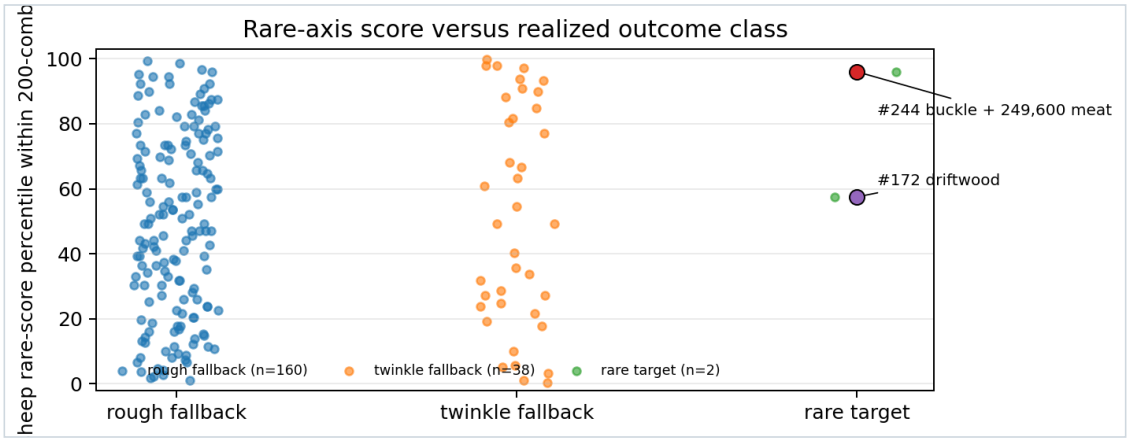
`screenshot sha256 6bd3d156cd17456858efca54…  html sha256 915fbe846376cca2c8bbb3f7…`

3.5 WaterSheep rare-axis checkpoint - 200 combs

The frozen run contains the first positive that is unusual on WaterSheep's rare axis before the outcome, while the comparison driftwood remains middle-of-pack. This is evidence worth following, not evidence of learned rare discrimination.



Figure 3-B. Native imagery from the frozen HTML ML Lab: pre-action utility trace and shadow ranking history.



Derived from the embedded event ledger in the same frozen HTML artifact; percentile uses within-cohort midranks.

event / realized loot	model utility	policy utility	rare mass	cohort ranks
#244 · beach 9452 buckle + 249,600 meat	0.34175	0.42175	**0.03910**	#32 / #32 / #8
#172 · beach 8540 driftwood	0.33681	0.41681	**0.03680**	#93 / #92 / #84

READING RULE / DO NOT OVERCLAIM

- Event #244 ranks #32/200 on model utility, #32/200 on policy utility, and #8/200 on rare score mass - upper-16% / top-4% behavior.
- Event #172 ranks #93, #92, and #84: middle-of-pack on the rare axis despite producing driftwood.
- Run means stay compressed: rough n=160, model 0.33661 / rare 0.03654; twinkle n=38, 0.33712 / 0.03665; rare n=2, 0.33928 / 0.03795.
- One of two rare targets is top-20 by rare score. With n=2 positives this can be luck; predictions are labeled UNVALIDATED_FOR_KOL_BEACH.

4 Case study: autoscend as an instrumented ascension engine

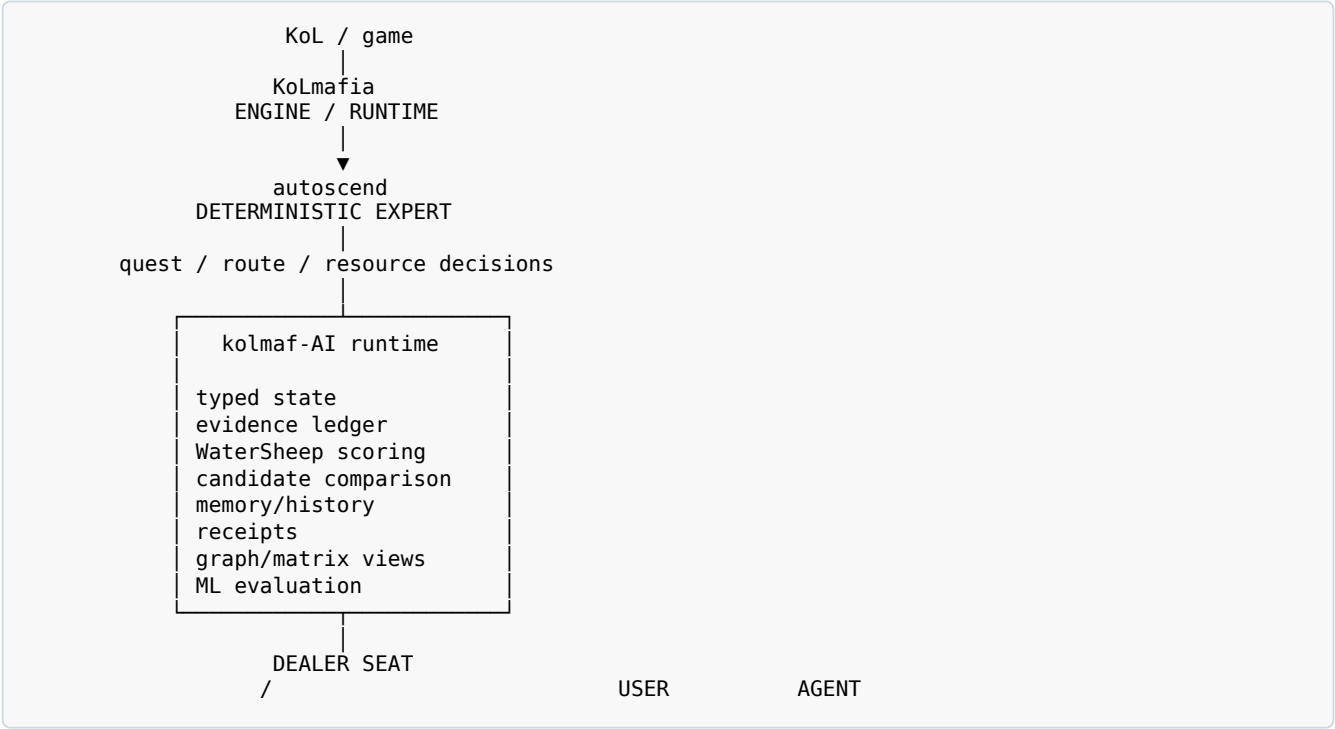
autoscend is a useful stress test for the idea because it is already the hard part: a mature deterministic ascension system with quest logic, path logic, combat, routing, equipment, familiars, acquisition, restoration, consumption, and pre/post-adventure behavior. The useful integration is not to replace that knowledge with an LLM. It is to expose its decision seams to the common runtime.

autoscend already owns ascension mechanics

```
KoLmafia
   │
   ▼
autoscend
├─ quest logic
├─ path logic
├─ combat
├─ routing
├─ equipment
├─ familiars
├─ acquisition
├─ restoration
├─ consumption
├─ pre-adventure setup
├─ post-adventure handling
└─ thousands of deterministic KoL rules
```

autoscend still knows HOW to ascend. Kolmaf-AI helps reason about WHAT autoscend is choosing, WHY, and what happened afterward.

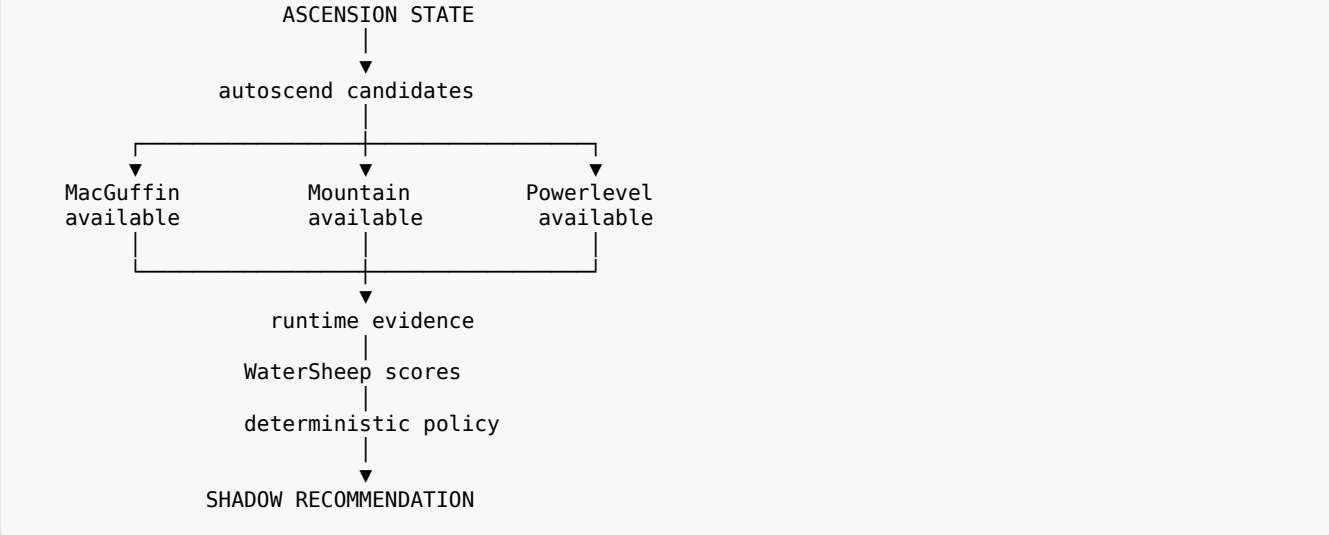
4.1 Wrap the expert, do not replace it



4.2 Routing becomes a measurable decision

At a routing boundary, autoscend may have several valid pieces of work available. The deterministic script still decides normally; the runtime freezes the same state, scores the exposed candidates, and records the disagreement without gaining authority.

```
possible work
├─ advance Level 11 MacGuffin
├─ work Level 10 mountain
├─ farm a required item
├─ powerlevel
├─ restore resources
└─ prep another quest
```




```
autoscend chose MacGuffin
          │
          ▼
actual result
├─ turns spent
├─ resources consumed
├─ quest progress
├─ items gained
├─ failures/retries
└─ resulting game state
          │
          ▼
bind result to the
PRE-ACTION prediction
```

4.3 Decision seams below whole quests

```
ASCENSION
   │
   ├── ROUTING
   │      ├─ quest A
   │      ├─ quest B
   │      └─ powerlevel
   │
   ├── RESOURCE USE
   │      ├─ spend item
```

```
    │     ├─ save item
    │     └─ acquire replacement
    │
    ├── EQUIPMENT
    │     ├─ loadout A
    │     ├─ loadout B
    │     └─ preserve resource
    │
    ├── FAMILIAR
    │     ├─ familiar A
    │     ├─ familiar B
    │     └─ utility familiar
    │
    ├── COMBAT PREP
    │     ├─ buffs
    │     ├─ restore
    │     └─ proceed
    │
    └── RECOVERY
          ├─ retry
          ├─ reroute
          ├─ ask user
          └─ stop safely
```

The learned layer should receive bounded candidate sets, not arbitrary execution primitives. The distinction is between assessing already-valid choices and inventing a new ascension procedure.

```
assess([
  ROUTE_MACGUFFIN,
  ROUTE_MOUNTAIN,
  POWERLEVEL
])

not

execute_arbitrary_ash()
```

4.4 Every ascension becomes an experiment stream

```
turn 101
  autoscend chooses route A
  WaterSheep preferred route B
        ↓
      record

turn 102
  resource decision
        ↓
      record

turn 130
  quest branch
        ↓
      record

turn 180
  recovery decision
        ↓
      record

turn 300
  equipment/resource tradeoff
```

```
      ↓
   record
```

```
ASCENSION RECEIPT

1,237 turns
412 meaningful decision points

routing disagreements       73
resource disagreements      18
equipment disagreements     41

when autoscend won          ...
when shadow scorer won      ...
indeterminate               ...

high-cost mistakes          ...
successful recoveries       ...
repeated state patterns     ...
```

A normal automation run becomes a machine-learning dataset generated by real gameplay. Phase one can remain entirely shadow-only: autoscend executes exactly as before while the seeded runtime watches, scores, and records.

4.5 Progressive trust, not a sudden autonomy jump

```
PHASE 1

autoscend
     │
     ├──────────────→ KoLmafia
     │                   executes normally
     │
     └──────────────→ shadow runtime
                         watches + scores

Zero ML authority.
```

```
LATER, IF EVIDENCE SUPPORTS IT

              deterministic autoscend
                        │
                proposes route
                        │
                        ▼
                runtime scorer
                        │
                ┌───────┴────────┐
                ▼                ▼
           agrees              disagrees
             │                   │
             ▼                   ▼
          proceed          log / surface
                              to dealer
```

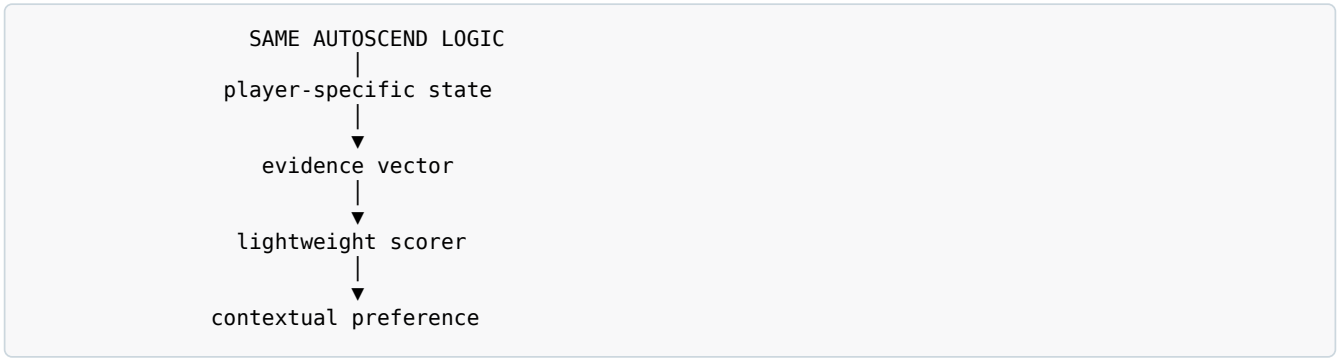
Even if a narrow decision eventually allows runtime reordering, autoscend should still define the legal candidate set, execution mechanics, cleanup, recovery, and hard safety rules.

4.6 Player variability becomes contextual evidence

```
player A
├─ 500 perms
├─ giant IotM collection
└─ tons of resources

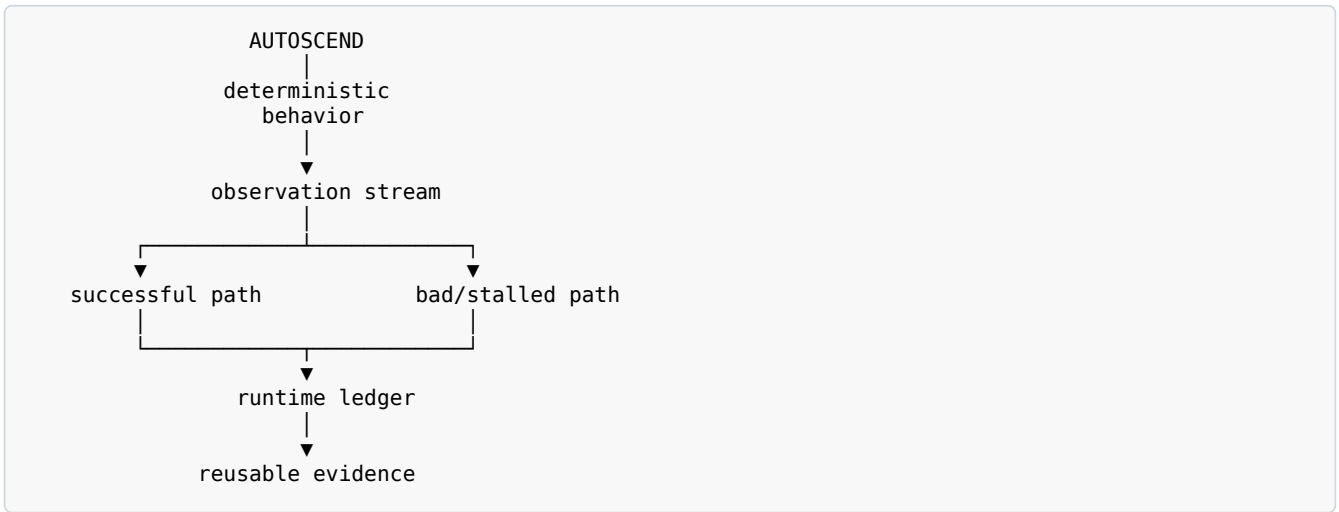
player B
├─ 20 perms
├─ a few IotMs
└─ very different optimal route

player C
├─ weird challenge path
├─ different resources
└─ different constraints
```

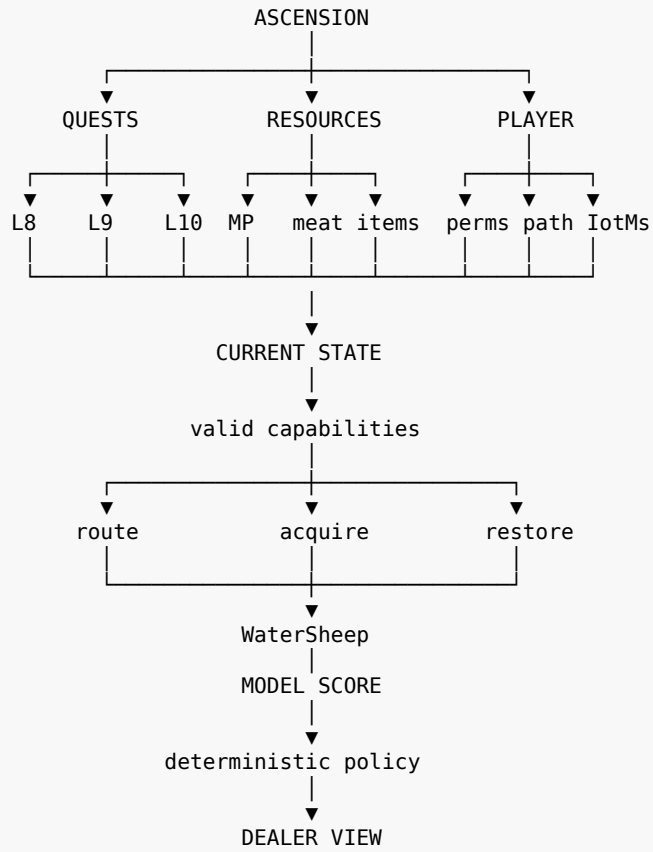


The bounded learned question is not ’invent an ascension route.’ It is ’given the routes the deterministic system says are valid, which historically fits this character and this state best?’

4.7 The deterministic system becomes teacher, baseline, and experiment generator



4.8 Graph / matrix / data for ascension debugging



For autoscend, the state space is no longer a million beach addresses. It is ascension state x capabilities x outcomes. The same representation pattern still applies: graph for relationships, matrix for structure/comparison, exact table for evidence, receipt for provenance.

4.9 Failure receipts instead of giant-log archaeology

```
FAILURE RECEIPT

phase:
  Level 11

current capability:
  MacGuffin route

pre-state:
  ...

candidate routes:
  5

deterministic selection:
  route_A

WaterSheep shadow:
  route_C

last successful transition:
  ...
```

```
repeated state:
  yes

progress delta:
  0 after 6 attempts

resources:
  ...

relevant evidence:
  ...

trace:
  decision #281 → #287
```

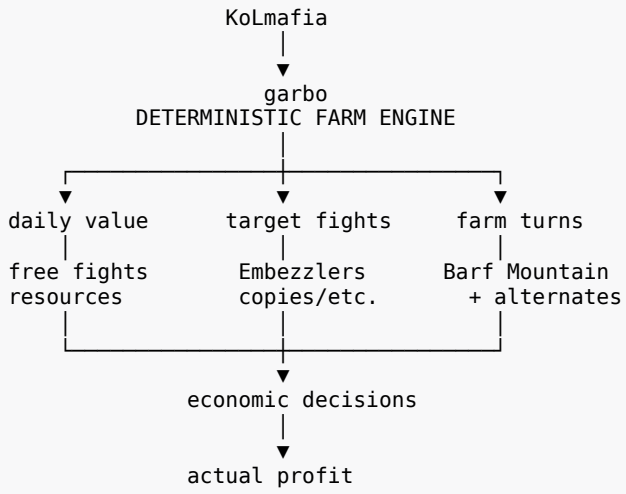
The same artifact becomes useful to a player, an autoscend developer, a coding agent, and an eventual dealer. OpenCode no longer needs to rediscover the entire script before it can answer 'why are we stuck?'

4.10 autoscend summary

Do not replace mature KoL automation with AI. Seed mature automation with a common intelligence/evidence runtime, let it generate structured experience, and leave execution authority with the tool that actually understands the domain.

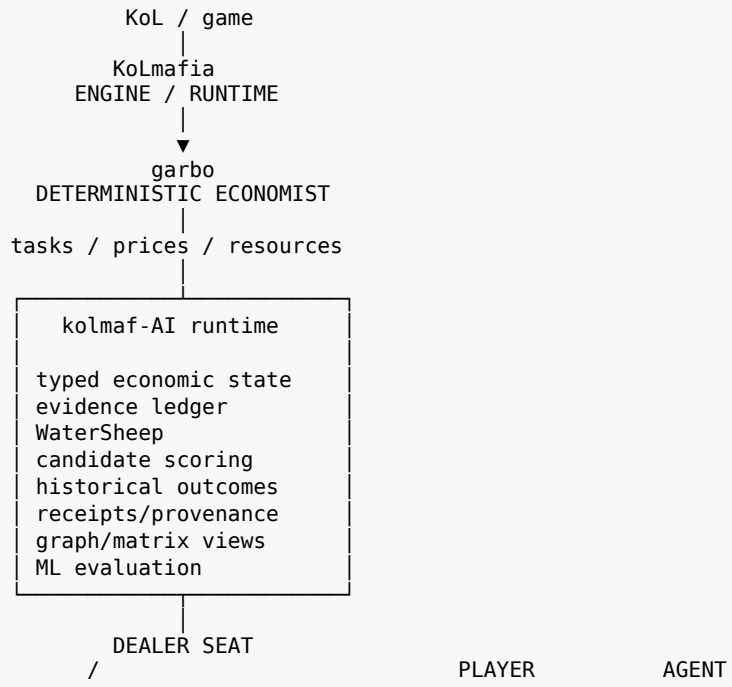
5 Case study: garbo as an ML wind tunnel for KoL economics

garbo makes the distinction even clearer because it already is an optimizer. Its purpose is to extract value from the player's available adventures and resources. The runtime does not need to invent meat-farming mechanics; it can measure, compare, and learn around the economic decisions garbo already makes.



Current garbo already reasons over valueOfAdventure, item and mall values, turn costs, opportunity costs, familiars, equipment, copy sources, free fights, diet, wanderers, target monsters, limited daily resources, and whether the player plans to ascend. It also has a structured Grimoire task engine and safe interruption boundaries.

5.1 Wrap the deterministic economist



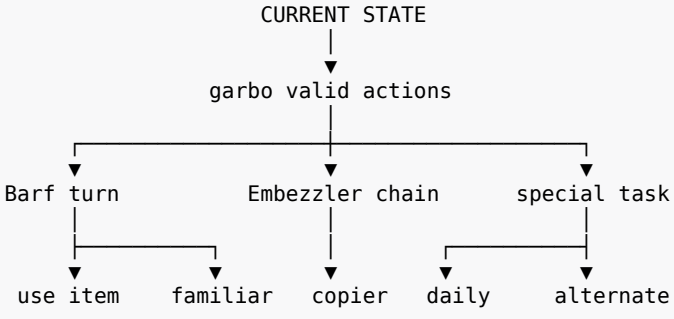
Authority remains explicit

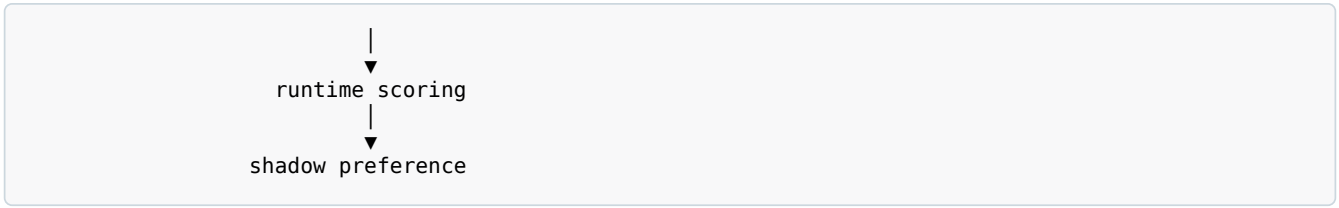
```
garbo       = knows HOW to farm
runtime     = observes and compares decisions
WaterSheep  = learned assessment
KoLmafia    = actual state + execution authority
dealer      = player / agent / dispatcher
```

5.2 Natural decision seams: ’is this worth doing?’

garbo already contains many decisions shaped like reward value versus adventure cost and resource opportunity cost. Those are ideal experimental seams because the legal alternatives are already known and the deterministic policy already exposes a baseline.

```
value(reward)
    >
turn cost × valueOfAdventure
    +
resource cost
```





5.3 Prediction versus realized value

```
GARBO:
"This action is worth 8,200 meat."

WATERSHEEP:
"Given the current state and historical outcomes,
I rank this action below these two alternatives."

GARBO:
"I'm still doing mine."

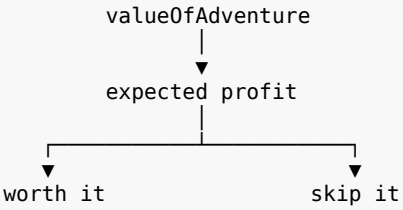
RUNTIME:
"Cool. Recording it."

                          ↓

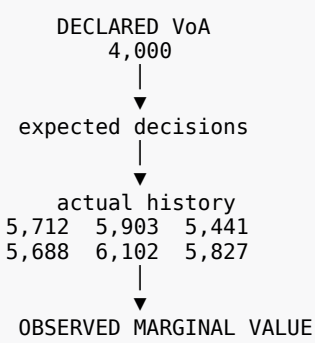
                  REAL OUTCOME
                          ↓
              prediction ↔ result
```

Combo's world is beachfront / tile / signal / loot. garbo's world is turn / resource / expected value / opportunity cost / actual profit. The same evidence contract still applies.

5.4 valueOfAdventure as a measurement target



Because valueOfAdventure influences so many downstream choices, the seeded runtime can compare the configured number against realized marginal return without silently rewriting it.



```
FACT:
player configured VoA = 4,000

MODEL:
recent behavior looks closer to 5,700

POLICY:
garbo continues using configured value

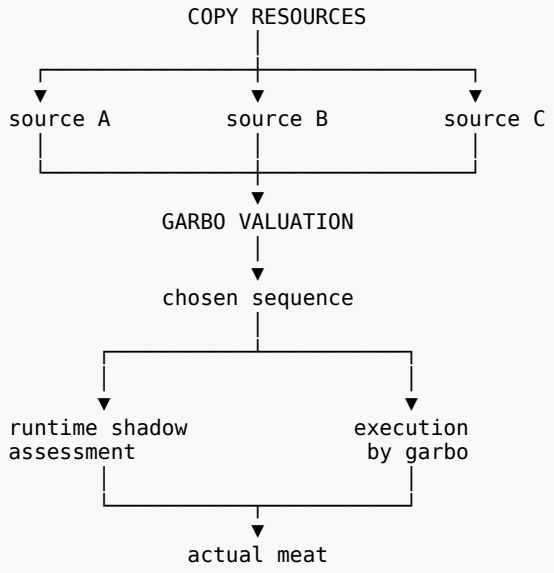
DEALER:
may choose to change it
```

5.5 Repeated economic decisions are excellent training/evaluation material

```
Is this resource profitable?
Is this turn worth spending here?
Is this familiar better?
Is this copier worth burning?
Is this item worth buying?
Is this daily resource worth consuming?
Is the opportunity cost acceptable?
```

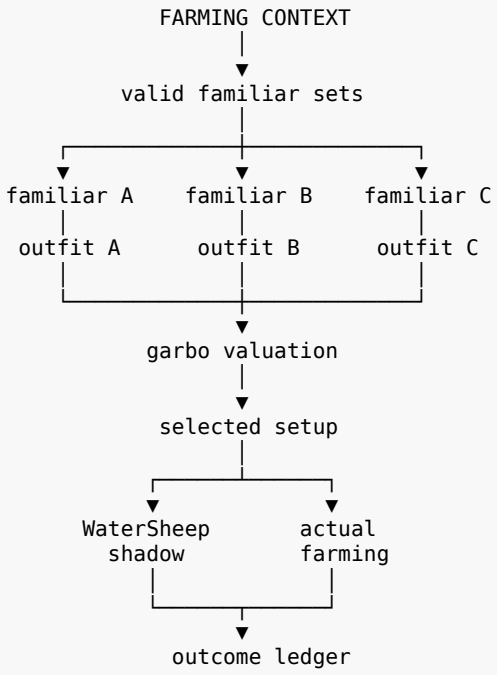
```
STATE
  ↓
CHOICE
  ↓
EXPECTED VALUE
  ↓
ACTUAL RESULT
```

5.6 Embezzler chains as a strategy experiment



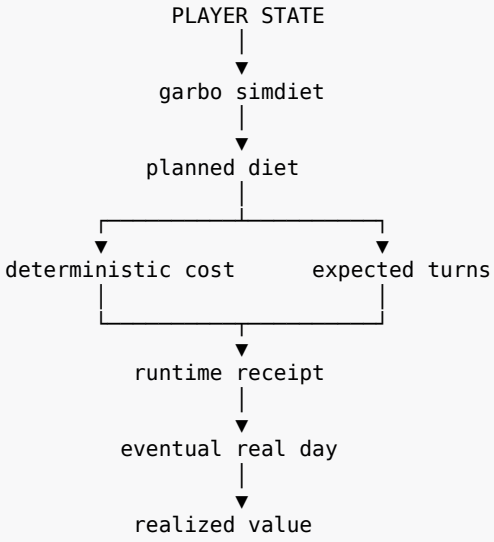
After enough runs, the corpus can surface which resources appear systematically overvalued or undervalued, which account states change the answer, and where deterministic expected value diverges from realized value.

5.7 Familiar and equipment choices: evaluate the optimizer, not replace it



The better question is not 'can AI build a better outfit?' It is 'across thousands of real farming turns, where do predicted and realized value systematically disagree?'

5.8 Diet becomes a shadow experiment




```
diet proposal
→ expected adventures
→ expected economic value
→ actual adventures
```

```
→ actual farming return
```

5.9 Decision receipts make complex failures legible

```
DECISION #2841

context:
  FARM / SOBER

garbo task candidates:
  Lucky Embezzler
  Yachtzee
  Barf Mountain
  Daily Dungeon

selected:
  Lucky Embezzler

reason:
  expected value > valueOfAdventure

inputs:
  VoA             5,500
  target value    21,300
  turn cost       1
  resource cost   8,200

WaterSheep shadow:
  rank 2 / 4

actual:
  +19,842 meat-equivalent

result:
  profitable

prediction error:
  -1,458
```

This receipt is valuable even if WaterSheep is removed. Instrumentation is not a side effect of ML; it is one of the primary products.

5.10 Garbo's task engine already resembles a capability envelope

```
ready
completed
do
outfit
combat
spendsTurn
sobriety
turns
```

A runtime adapter can project these structured task concepts into a common envelope without asking an agent to comprehend the entire repository on every decision.

```
TASK
  name
  ready
  expected turns
  expected value
  resource costs
  relevant state
```

```
        ↓

COMMON CAPABILITY ENVELOPE
```

```
    capability: "garbo.nonbarf.daily-dungeon",

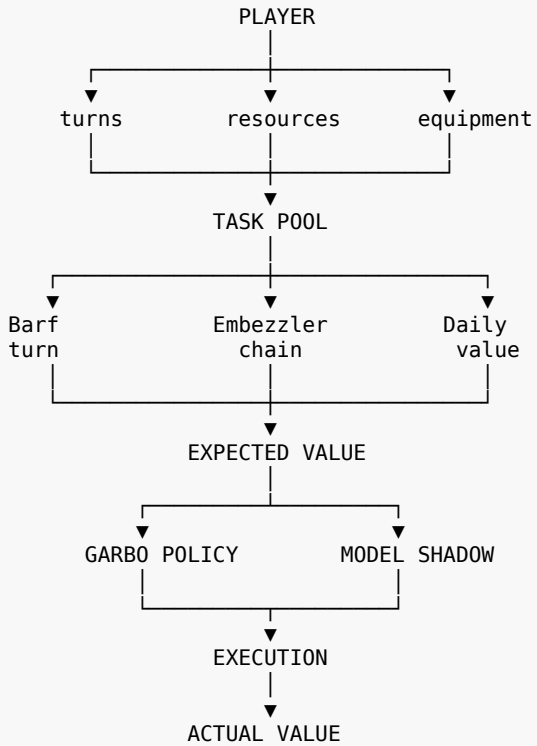
    state: ...,

    expected:
    turns: 3,
    reward_value: ...,
    opportunity_cost: ...
    ,

    authority: "garbo",

    alternatives: [...]
```

5.11 Economic graph / matrix / data




```
rows    = capability/task families
columns = economic/resource dimensions

           turn   meat   item   copier   familiar   daily   risk
Barf       ▆▆     ▆▆     ▆▆       ·      ▆▆         ·       ▆
Embezzler  ▆      ▆▆▆    ▆        ▆▆     ▆▆         ▆       ▆
Yachtzee   ▆▆     ▆      ▆▆       ▆▆     ▆▆         ▆       ▆▆
Daily      ▆      ▆▆     ▆▆       ·      ▆          ▆▆      ▆
```

The same visualization rule used in the Combo proof surface applies: graph = relationships,

matrix = structure/cross-comparison, table = exact values, receipt = provenance.

5.12 Detect valuation drift without letting ML override

```
EXPECTED
9,500

OBSERVED
7,310 ± ...

MODEL
persistent negative residual

STATUS
valuation may be stale
```

The learned sidecar can identify persistent residual patterns and surface an assumption for inspection. That is not proof that garbo is wrong; it is a high-value signal for a developer or user to investigate.

5.13 ML as a spading instrument

```
garbo
  ↓
real play
  ↓
runtime evidence
  ↓
ML detects pattern
  ↓
developer investigates
  ↓
deterministic garbo improvement
  ↓
more real play
```

If a learned pattern becomes verified game knowledge, the best outcome may be to absorb that truth into deterministic code and remove the learned component from that decision path.

5.14 The dealer can remain thin

```
Dealer:
"What's garbo doing?"

Runtime:
"Current task: Barf turn.
Three higher-level alternatives are available.
Garbo ranks Barf #1.
WaterSheep ranks it #2.
No anomaly detected."

Dealer:
"Continue."
```

```
WaterSheep cannot:

buy arbitrary stuff
spend arbitrary resources
choose arbitrary URLs
construct arbitrary combat
```

```
rewrite task validity
change KoL state directly
```

```
SCORE(
  Barf,
  Embezzler,
  Yachtzee,
  Daily Dungeon
)
```

garbo still says which options are legal, their deterministic costs, how they execute, and how failures recover. KoLmafia remains the state authority.

5.15 garbo summary

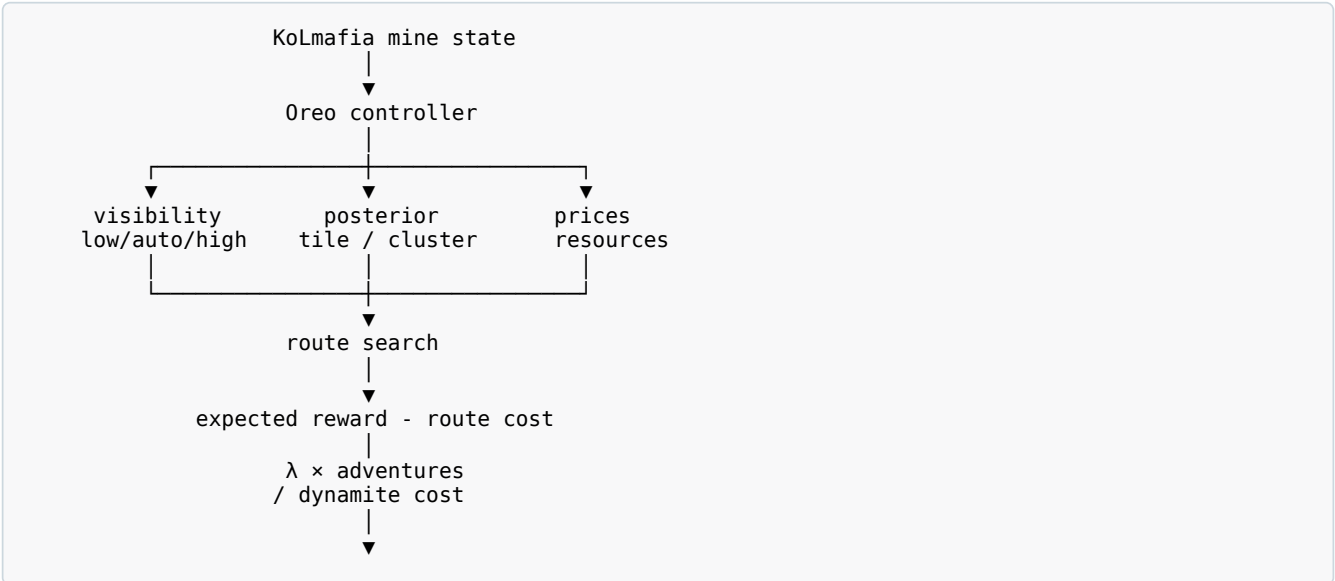
Combo became an ML wind tunnel for Beach Comb. garbo could become an ML wind tunnel for KoL economics: an instrumented deterministic farming engine whose decisions can be observed, compared, tested, and improved from real evidence.

6 Case study: Oreo as a calibration laboratory

Oreo is the cleanest example of a further step because it already contains an explicit mathematical model. In the design notes summarized here, Oreo is not simply a sparkle-following miner: it combines visibility, posterior reasoning, price/resource inputs, route search, expected reward, route cost, lambda, dynamite decisions, and mine/reset behavior.

Scope of this case study

The Oreo chapter below is based on the project design material supplied for this working paper. It should be read as a proposed integration pattern around the described Oreo controller and strategies, not as a claim about an upstream public interface unless separately documented by the Oreo project itself.



```
                  MINE or RESET
```

6.1 Oreo already has purpose-built strategy models

```
pjb
   deterministic community-style sparkle mining

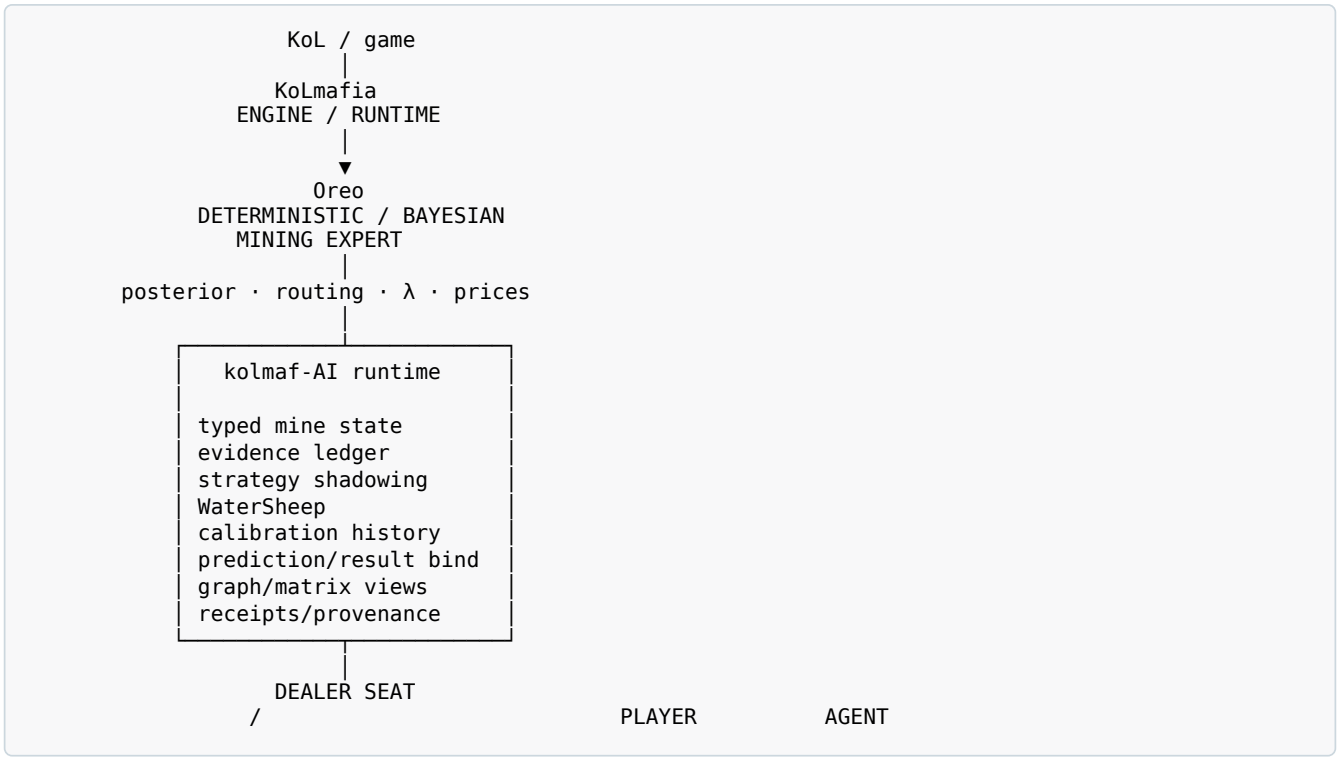
oreo
   PJB + longest second-row-vein opening move

ev
   per-tile posterior EV

ev-cluster
   connected six-tile velvet posterior
```

The wrong integration would be to ask WaterSheep to replace this purpose-built optimizer. The useful integration is to instrument the optimizer, shadow multiple strategies on the same frozen state, and measure where assumptions match reality.

6.2 Instrument the optimizer itself



6.3 A nearly perfect experiment boundary

```
Decision
├─ mine coordinate
│    └─ reason
│
└─ reset
     └─ reason
```

Before each action, the runtime can freeze mine layout, opened tiles, observed sparkle/-

dull/resource evidence, visibility mode, Object Detection state, dynamite price/availability, resource values, second-gold assumptions, lambda, and strategy. Only after the prediction is frozen does Oreo execute.

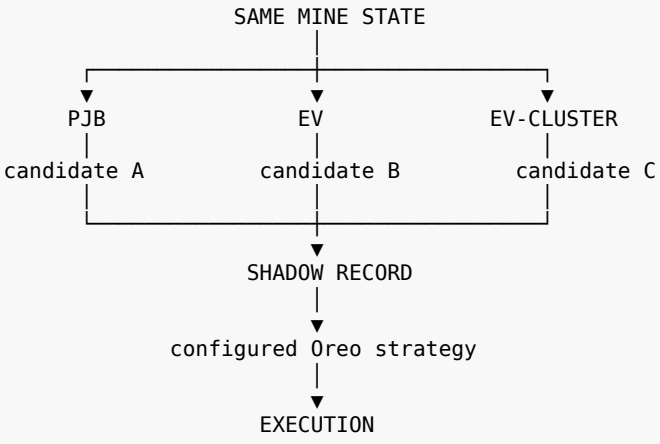
```
PRE-ACTION STATE

mine layout
opened tiles
known sparkles
known dull tiles
observed ore/gold/crystal
visibility mode
Object Detection state
dynamite price
dynamite available
resource values
second-gold assumption
λ
strategy
```

```
STATE
 ↓
PREDICTION
 ↓
DETERMINISTIC ACTION
 ↓
OBSERVATION
 ↓
prediction ↔ actual result
```

The question is sharper than Combo's. Oreo can be evaluated not only on whether a tile was valuable, but on whether its expected-value model was calibrated.

6.4 Shadow all strategies on one frozen state



Only the configured Oreo strategy receives authority. One-step alternatives can be compared directly from the same real state. Longer alternate futures must be labeled as simulation because choosing a different tile would change subsequent state.

Provenance classes

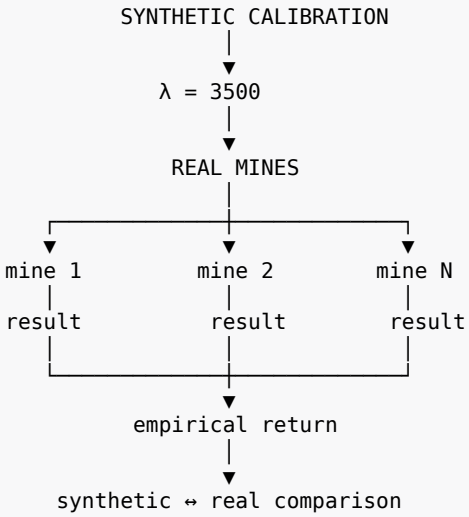
```
FACT
actual executed trajectory

COUNTERFACTUAL
one-step unexecuted alternative

SIMULATION
synthetic alternate trajectory
```

6.5 Lambda becomes a measurable policy parameter

Oreo's EV strategy can be understood as expected resource value minus the cost to reach it, where route cost may include lambda times adventures or a dynamite substitution cost. The runtime can connect synthetic calibration to real outcomes.



The interesting question is whether the lambda that wins on synthetic boards also wins on actual mines.

6.6 Measure model assumptions instead of replacing them

```
MODEL ASSUMPTION

P(second gold)
   0.496

REAL CORPUS

observed second-gold frequency
   0.???

confidence
   ...

sample size
   ...
```

The same analysis applies to ore geometry, connected-cluster assumptions, and visibility. The runtime can produce posterior residuals and reveal which mine-state regimes are well

calibrated or data-starved.

6.7 WaterSheep as a model-of-the-model

```
OREO:
tile [4,3] EV = 7,914
confidence comes from deterministic posterior

WATERSHEEP:
historical pattern class =
  MODEL_WELL_CALIBRATED  0.68
  MODEL_OVERCONFIDENT    0.21
  INDETERMINATE          0.11
```

```
CHOICE

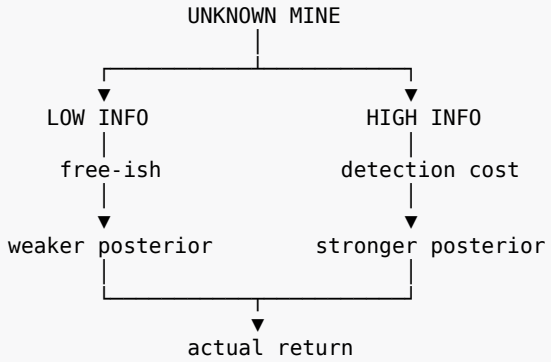
Which strategy historically behaves best
in states resembling this one?

EV              0.21
EV_CLUSTER      0.58
OREO            0.15
PJB             0.06

authority = none
```

This is where a learned model has a genuine comparative advantage: high-dimensional historical pattern recognition. Oreo remains responsible for exact rules, legality, pathfinding, posterior math, resource accounting, and execution.

6.8 Visibility becomes an information-economics experiment

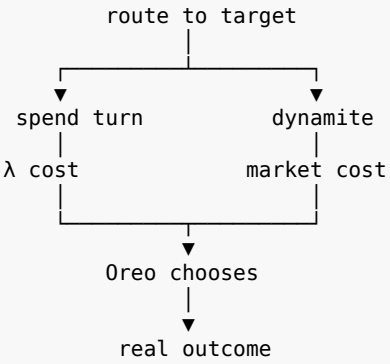


The runtime can measure whether the cost of additional information actually changes route decisions, reset timing, and realized value.

```
HIGH VISIBILITY RECEIPT

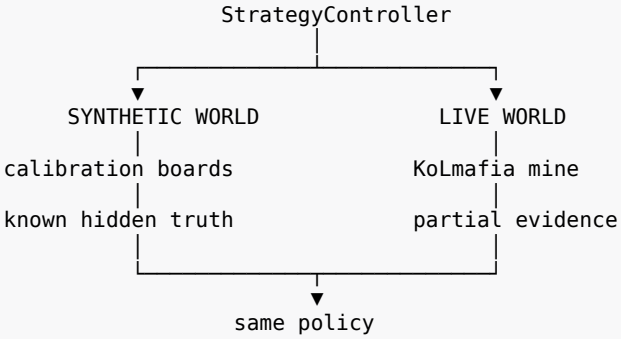
detection uses          87
expected information EV ...
actual additional EV    ...
changed first decision  41%
changed reset timing    19%
net observed benefit    ...
```

6.9 Dynamite becomes a cost-substitution experiment



A persistent residual does not authorize the learned layer to override the choice. It surfaces an assumption worth examining.

6.10 Offline and live worlds can share the same policy contract



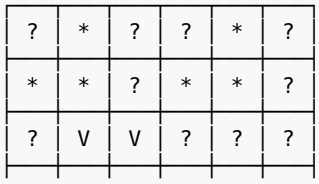

```
OFFLINE

10,000 deterministic synthetic mines
      ↓
strategy behavior
      ↓
expected calibration

LIVE

500 actual mines
      ↓
same strategy contract
      ↓
real calibration
```

6.11 The mine itself is a proof surface



```
┌───┬───┬───┬───┬───┬───┐
│ ? │ V │ V │ V │ ? │ G │
├───┼───┼───┼───┼───┼───┤
│ · │ · │ · │ · │ · │ · │
├───┼───┼───┼───┼───┼───┤
│ O │ O │ · │ · │ · │ · │
└───┴───┴───┴───┴───┴───┘
```

```
TILE [4,3]

FACT
  sparkle observed
  legal
  2-step route

OREO MODEL
  P(velvet)       .43
  P(gold)         .07
  P(crystal)      .11
  expected value  5,941

POLICY
  route cost      3,500
  score           +2,441

WATERSHEEP
  calibration confidence .72

ACTION
  selected by ev-cluster

OUTCOME
  unsmoothed velvet
```

This is not decorative visualization. It is a proof surface for a probabilistic optimizer: FACT, mathematical MODEL, deterministic POLICY, learned meta-model, ACTION, and OUTCOME remain separately legible.

6.12 Calibration, not single-event correctness

A probabilistic model is expected to be wrong on individual events. The test is whether predicted frequencies match realized frequencies over enough comparable observations.

```
predicted gold probability

0–10%      → actual frequency ?
10–20%     → actual frequency ?
20–30%     → actual frequency ?
...

classification:
CALIBRATED
OVERCONFIDENT
UNDERCONFIDENT
DATA-STARVED
```

6.13 Model residuals can become deterministic knowledge

```
Oreo
 ↓
deterministic posterior
 ↓
real gameplay
```

```
 ↓
structured residuals
 ↓
pattern analysis
 ↓
human verification
 ↓
deterministic Oreo improvement
```

```
ML discovers candidate pattern
          ↓
human verifies mechanic
          ↓
deterministic code absorbs truth
```

6.14 Structured explainability becomes provenance

```
DECISION #381

strategy:
  ev-cluster

visibility:
  high

state:
  cavern 17
  opened 8 / 36

candidate:
  [4,3]

model:
  expected reward 5910

route:
  steps 2

policy:
  lambda 3500
  route cost 3500
  final score +2410

dynamite:
  available true
  used false

authority:
  oreo

result:
  velvet

actual value:
  ...

shadow:
  ev chose [2,4]
  oreo chose [3,1]
  pjb chose [1,2]

WaterSheep:
  model-calibration confidence ...
```

6.15 The dealer remains a reader/caller, not a miner implementation

```
Agent:
"Why does Oreo want to reset?"

Runtime:
"Best remaining route has expected reward 6,410
against route cost 7,000.

EV-cluster recommends RESET.
EV also recommends RESET.
Community Oreo would probe [3,1].
No calibration anomaly detected."
```

Authority stack

```
KoLmafia
   = world and game-state authority

Oreo
   = mining-domain authority

Oreo posterior / EV
   = deterministic probabilistic model

WaterSheep
   = learned shadow / meta-model

kolmaf-AI runtime
   = evidence + instrumentation + comparison

Dealer
   = chooses which supported capability to invoke

Player / agent
   = occupies or supervises dealer seat
```

6.16 Oreo summary

```
Combo:

deterministic beach script
        +
runtime seed
        =
ML wind tunnel for Beach Comb


garbo:

deterministic economic optimizer
        +
runtime seed
        =
ML wind tunnel for KoL economics


Oreo:

probabilistic mining optimizer
        +
runtime seed
        =
ML wind tunnel for model calibration
```

With Oreo, the learned layer does not primarily ask 'can you make a better decision?' It asks 'when is the existing mathematical decision model well calibrated, when are its assumptions drifting, and where are the interesting residuals?'

7 The common runtime contract

The four examples point toward a common envelope that can be implemented without making every capability look identical internally. The domain tool contributes semantics; the shared runtime contributes evidence discipline and a consistent surface for users, models, and future dealer adapters.

7.1 The invariant loop

```
STATE
  │
  ▼
CANDIDATES
  │
  ├──────────────────► MODEL / SHADOW
  │
  ▼
DETERMINISTIC POLICY
  │
  ▼
AUTHORIZED CAPABILITY ACTION
  │
  ▼
RESULT / READBACK
  │
  ▼
RECEIPT + LEDGER
```

7.2 A minimal capability envelope

```
capability
  id
  version
  authority_owner

state
  authoritative inputs
  freshness / state binding

evidence
  facts
  provenance
  uncertainty

candidates
  legal options
  deterministic metadata

model
  optional shadow scores
  calibration state
  authority = none

policy
  deterministic ranking / gates
```

```
action
  capability-owned invocation

result
  independent readback

receipt
  pre-state
  prediction
  policy decision
  actual outcome
  hashes / timestamps / trace IDs
```

7.3 FACT, MODEL, POLICY, ACTION, OUTCOME

Plane	Meaning	Authority
FACT	Observed or authoritative state/evidence	Can constrain decisions; not automatically permission
MODEL	WaterSheep or other learned assessment	Advisory by default
POLICY	Deterministic rules, valuation, ranking, gates	Capability-specific decision logic
ACTION	The actual supported operation	Owned by the capability/KoLmafia path
OUTCOME	Independent post-action observation	Ground truth for evaluation and readback

7.4 WaterSheep should be replaceable

WaterSheep is useful precisely because it is small enough not to dictate the architecture. A capability should be able to substitute AnyJev, another local model, a remote model, an ensemble, or a handwritten scorer without changing its authority model. The interface is the important asset; the current learned primitive is one implementation.

7.5 Runtime seed versus domain adapter

Shared runtime provides	Domain capability provides
Evidence ledger and provenance	Meaning of state fields
Prediction/result binding	Legal candidate generation
Model adapter and calibration history	Deterministic policy / valuation
Receipts, run control, history	Actual action implementation
Graph/matrix/table proof surfaces	Domain-specific visualization semantics
Dealer-facing capability contract	Cleanup, recovery, and hard safety rules

8 Migration path from the public desktop direction

The new architecture does not require a flag day. It can grow out of the current tool constellation by moving stable contracts toward the capabilities that can actually use them.

8.1 1. Preserve the current trust model

Keep KoLmafia as runtime truth, model output as proposal material, explicit human/agent boundaries, and independent readback after mutation. These are already core strengths of the public project direction.

8.2 2. Treat Combo as the reference seed

Extract the reusable parts of the Combo/WaterSheep loop: state envelope, evidence typing, candidate contract, model adapter, prediction/result pairing, receipts, run-control, proof surface, and artifact export. Domain-specific Beach Comb semantics remain in Combo.

8.3 3. Add read-only/shadow adapters to mature tools

Start with adapters that expose state and candidate decisions without changing execution. autoscend routing, garbo economic choices, and Oreo strategy/calibration decisions are different domains but can share the same outer lifecycle.

8.4 4. Standardize dealer discovery

```
list_capabilities()
inspect(capability)
propose(capability, state)
execute(capability, approved_request)
read_receipt(trace_id)
```

The dealer API should remain thin and should not become a generic arbitrary-command tunnel.

8.5 5. Let OpenCode arrive late

An OpenCode plugin is easier and safer once capabilities already speak a stable contract. At that point OpenCode becomes a discover/inspect/propose/call/read-receipt adapter rather than the place where KoL-specific execution behavior must be reinvented.

8.6 6. Promote learned authority only by evidence, if ever

Shadow mode should be the default. Any transition from model observation to model-influenced ordering should be narrow, measured, reversible, and bounded by capability-defined legal options. A model should not gain generic action authority simply because it performed well on one task family.

9 What this reframing buys

- Cross-deployment through a shared runtime contract instead of duplicated large bridges.
- Reuse of mature community automation as deterministic experts rather than replacement targets.
- A common evidence language that can be consumed by humans, developers, WaterSheep, or a future agent.
- Structured experiment data generated by normal play instead of separate synthetic-only ML projects.
- Debugging artifacts that remain useful even if the learned model is removed.
- A path for learned discoveries to become verified deterministic rules.
- Thin agent integration: discovery and capability calls instead of broad arbitrary execution.
- A clearer place to keep authority: with KoLmafia and domain tools, not with inference.

10 Closing thesis

The important change is conceptual. Kolmaf-AI does not need to grow into a monolithic KoL-playing agent surrounded by more and more bridges. It can become a portable instrumentation and intelligence substrate that plugs into the automation ecosystem already doing the hard domain work.

```
ENGINE UNDERNEATH

KoLmafia

        ↓

INTELLIGENCE / TOOLING IN THE MIDDLE

kolmaf-AI runtime seed
FACT · MODEL · POLICY · receipts · memory · visualization

        ↓

CAPABILITIES THAT KEEP THEIR KEYS

Combo · autoscend · garbo · Oreo · future ASH/JS tools

        ↓

REPLACEABLE DEALER ON TOP

human · deterministic dispatcher · OpenCode · future agent
```

Do not make AI the game client. Make mature KoL capabilities observable, comparable, model-readable, and composable - while leaving execution authority with the engine and the domain tool that actually understands the operation.

11 References and source notes

1. **Kolmaf-AI public project page.** https://doncannoli-burns.github.io/kolmaf-ai-desktop/. Accessed 12 October 2026. Used for the prior-direction context: Linux-first desktop/control-desk framing, KoLmafia as runtime truth, LLM proposal role, human gating, context tools, policy gates, readback, and companion-tool constellation.

2. **KoLmafia.** https://github.com/kolmafia/kolmafia. Runtime and scripting foundation referenced throughout.

3. **autoscend.** https://github.com/loathers/autoscend. The autoscend chapter is a proposed runtime-seed integration around a mature deterministic ascension system; it does not claim upstream autoscend currently implements the described Kolmaf-AI hooks.

4. **Garbage Collector (garbo).** https://github.com/loathers/garbage-collector. Current repository reviewed for its value-maximization framing, valueOfAdventure, structured Grimoire task engine, safeInterrupt boundaries, non-Barf value checks, and Lucky/Embezzler task structure. The seeded ML/runtime layer is proposed, not current upstream functionality.

5. **Combo / WaterSheep working prototype.** Project development material from the

Kolmaf-AI working sessions, including the instrumented Beach Comb runtime, prediction/result binding, receipts, ML Lab, and linked graph/matrix/data proof surface.

6. **Oreo integration notes.** Project design material supplied in the working session for this paper. The chapter is a conceptual architecture case study around the described deterministic/Bayesian mining controller and strategy model.

12 Concept index

Index

adapter, 30
architectural inversion, 1
ascension graph, 11
ascension receipt, 8
authority dialogue, 5
authority invariants, 1
autoscend, 6
autoscend authority, 6
autoscend decision seams, 7

benefits, 31

calibration curve, 26
capability envelope, 29
capability seed, 4
closing thesis, 32
Combo, 4
common runtime contract, 29
contextual preference, 10
cross-deployment, 3

dealer seat, 2
dynamite, 25

economic decision seams, 13
economics drift, 19
Embezzler, 15
evidence planes, 30
expected vs actual, 14
explain provenance, 27

failure receipt, 11
familiar equipment, 16

garbo, 12
garbo authority, 13
garbo dealer, 19
garbo matrix, 18
garbo receipt, 17
GarboTask, 17

lambda, 23

meta-model, 24
migration, 30
mine decision, 21

offline online, 25
OpenCode, 5
Oreo, 20
Oreo dealer, 28
Oreo instrumentation, 21
Oreo matrix, 25
Oreo spading, 26
Oreo strategies, 21

posterior calibration, 23
prior direction, 1

references, 32
repeated decisions, 15
routing, 7
runtime seeding, 1

shadow mode, 9
simdiet, 16
spading, 19
state candidate decision result receipt, 29
strategy shadowing, 22

teacher baseline, 10

Unreal analogy, 3

valueOfAdventure, 14
visibility, 24

WaterSheep, 30